Nutan Maharashtra Vidya Prasarak Mandal's (NMVPM's)
NUTAN MAHARASHTRA INSTITUTE OF ENGINEERING AND TECHNOLOGY (NMIET)
An Autonomous Institute
Under Administrative Support – Pimpri Chinchwad Education Trust (PCET)
Approved by AICTE | Accredited by NAAC | Affiliated to SPPU

Technical Seminar Report on

CHAINGUARD V2: A TAMPER-EVIDENT LAST-MILE LOGISTICS CONTROL PLANE FOR SEALED DISPATCH, CUSTOMER-AUTHORISED CUSTODY BREAKS AND UNIT-EXACT SHRINKAGE RECONCILIATION

Project Reference: ChainGuard v2.4

SUBMITTED TO THE DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

SUBMITTED BY

Student Name	Roll No.
Ms. Pallavi Kokate	B34
Mr. Anil Kewat	B45
Ms. Nikita Sagar	B50

Under The Guidance of
Miss. Sakshi Ambadkar

ACADEMIC YEAR : 2026–27

Samarth Vidya Sankul, Vishnupuri, Talegaon Dabhade, Maharashtra 410507

Nutan Maharashtra Vidya Prasarak Mandal's (NMVPM's)

NUTAN MAHARASHTRA INSTITUTE OF ENGINEERING AND TECHNOLOGY (NMIET)

An Autonomous Institute

Under Administrative Support – Pimpri Chinchwad Education Trust (PCET)

Approved by AICTE **Accredited by NAAC** **Affiliated to SPPU**

CERTIFICATE

This is to certify that the technical seminar report

"ChainGuard v2: A Tamper-Evident Last-Mile Logistics Control Plane for Sealed Dispatch, Customer-Authorised Custody Breaks and Unit-Exact Shrinkage Reconciliation"

Submitted by

Student Name	Roll No.
Ms. Pallavi Kokate	B34
Mr. Anil Kewat	B45
Ms. Nikita Sagar	B50

Has completed the technical seminar report satisfactorily in the subject of Technical Seminar (ELC342CSE) for the academic year 2026–2027 as prescribed in the syllabus of Savitribai Phule Pune University, Pune.

The work was carried out under the guidance of Miss. Sakshi Ambadkar and has not been submitted earlier to any other institute or university for the award of a degree.

Miss. Sakshi Ambadkar	Dr. Mahesh Wankhede	Dr. Pramod Patil
Seminar Guide	**HOD**	**Principal**
Dept. of CSE	**Dept. of CSE**	**NMIET Pune**

Place : Pune

Date : _____

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to our seminar guide, Miss. Sakshi Ambadkar, for her continuous guidance, patient review and constructive criticism throughout the preparation of this report. Her questions repeatedly pushed us to separate what the prototype actually demonstrates from what we hoped it would demonstrate.

I am thankful to the faculty members and the Department of Computer Science and Engineering at NMIET for providing the infrastructure, laboratory access and academic freedom needed to design, build and deploy this project. I also acknowledge the open-source communities behind React, TypeScript, Vite, Tailwind CSS, Zustand and the Python libraries used to prepare this document; their documentation made a small, independent implementation possible.

Finally, I extend my heartfelt thanks to my family, teammates and peers for their constant motivation, honest feedback and support during the completion of this seminar. The working prototype, the deployment and the demonstration dataset were built collaboratively, and the credit for anything that works belongs to the team.

Pallavi Kokate (B34)
Anil Kewat (B45)
Nikita Sagar (B50)
(T.Y. B.E., NMIET)

ABSTRACT

Last-mile shrinkage is usually investigated at the warehouse, after the vehicle has returned and the count has failed. This report argues that the failure begins earlier, at dispatch, when a buffer quantity leaves the warehouse without an enforceable relationship to the seal, the driver, the route and the evidence of what happened in transit. The result is a custody gap: the final count exists but has no authority, because the conditions for judging it were never recorded.

The report presents ChainGuard v2, a browser-delivered control plane that makes the custody of a last-mile consignment explicit from dispatch to warehouse return. Each consignment is bound to a unique digital seal and a declared buffer quantity. The seal can be broken only through a customer-issued single-use unlock token, and the exact buffer count must return. Driver telemetry, GPS-anchored incidents, the customer attestation and the warehouse count share one dispatch record, so reconciliation compares fields within a record rather than reconstructing four separate systems.

The prototype implements four role portals — vendor dispatch, driver telemetry, customer verification and audit reconciliation — over a single reactive Zustand store. It formalises custody as a guarded state machine, computes a unit-exact variance, raises a priority-1 shrinkage alert and locks the driver's settlement on any non-zero variance, and derives fleet KPIs and a driver trust score from the same records. A seeded fleet of six dispatches covers every lifecycle state, and a reproducible fraud scenario demonstrates the complete path from seal issuance to payout lock.

The report also states the prototype's limits. The trust boundary is the browser, identifiers are not cryptographic secrets, settlement is a flag rather than a payment instruction, and the audit log is not yet immutable. These limitations define a concrete future scope: a server-side custody service, signed tokens and hardware seals, a hash-chained evidence log and integration with settlement systems. The broader contribution is a reusable custody pattern, not a product: one active seal, consent before break, buffer as expected return and variance as a settlement gate.

TABLE OF CONTENTS

Certificate ..... **I**
Acknowledgement ..... **II**
Abstract ..... **III**
Table of Contents ..... **IV**
List of Figures ..... **V**
List of Tables ..... **V**
List of Abbreviations ..... **VI**

1. Introduction 1
   1.1 Background 1
   1.2 Motivation 1
   1.3 Problem Statement 1
   1.4 Objectives 1
   1.5 Scope of the Seminar 2
2. Literature Survey 3
   2.1 Existing Work and Related Studies 3
   2.2 Summary of Research Papers 3
   2.3 Research Gaps 4
3. Overview of the ChainGuard v2 Control Plane 5
   3.1 Definition and Concept 5
   3.2 Architecture / Block Diagram 5
   3.3 Key Components and Technologies 6
4. Working / Methodology 8
   4.1 Working Principle 8
   4.2 Algorithms / Techniques Used 9
   4.3 Tools and Platforms 11
5. Applications 12
   5.1 Real-World Use Cases 12
   5.2 Case Study 12
6. Advantages and Limitations 13
   6.1 Advantages 13
   6.2 Limitations / Challenges 13
7. Comparison with Existing Technologies 14
8. Future Scope 15
9. Conclusion 16
REFERENCES 17

LIST OF FIGURES

Figure No	Name	Page no
Figure 3.1	Four-plane architecture of the ChainGuard v2 control plane	5
Figure 3.2	Custody lifecycle state machine, guarded transitions and settlement branches	6
Figure 4.1	End-to-end operational workflow across the four role portals	8
Figure 4.2	Warehouse reconciliation decision flow	10

LIST OF TABLES

Table No	Name	Page no
Table 2.1	Summary matrix of the literature survey	4
Table 3.1	Custody states, guards and permitted transitions	7
Table 4.1	Technology stack of the implemented prototype	11
Table 4.2	Seeded demonstration fleet (six dispatches)	11
Table 7.1	Feature matrix comparison with existing approaches	14

LIST OF ABBREVIATIONS

Abbreviation	Full Form
API	Application Programming Interface
CDN	Content Delivery Network
CI/CD	Continuous Integration / Continuous Delivery
CSPRNG	Cryptographically Secure Pseudo-Random Number Generator
CSV	Comma-Separated Values
DOM	Document Object Model
DPDP	Digital Personal Data Protection (Act, 2023)
GPS	Global Positioning System
HMAC	Keyed-Hash Message Authentication Code
HTTP	Hypertext Transfer Protocol
INR	Indian Rupee
IoT	Internet of Things
JSON	JavaScript Object Notation
JWT	JSON Web Token
KPI	Key Performance Indicator
NIST	National Institute of Standards and Technology
P1	Priority One
POD	Proof of Delivery
QR	Quick Response
RBAC	Role-Based Access Control
REST	Representational State Transfer
SDK	Software Development Kit
SPA	Single-Page Application
SRS	Software Requirements Specification
TLS	Transport Layer Security
TMS	Transport Management System
UI	User Interface
URL	Uniform Resource Locator
UTC	Coordinated Universal Time
UUID	Universally Unique Identifier
W3C	World Wide Web Consortium
XSS	Cross-Site Scripting
ZTA	Zero Trust Architecture

Chapter 1

1. Introduction

1.1 Background

Last-mile delivery is the least observable segment of the logistics chain. A consignment may pass through five hands between dispatch and the customer, yet most operators record only two events: the dispatch note and the proof of delivery. The interval between them is where shrinkage, damage disputes and settlement conflicts originate [1], [3]. The buffer stock carried to absorb doorstep damage is not counted at all until it returns, so a missing unit at the warehouse cannot be distinguished from a legitimate customer swap.

ChainGuard v2 makes that custody explicit. One record binds the consignment to a digital seal, a declared buffer quantity, a customer-authorised break, a telemetry trail and a unit-exact return count. The final count is judged against the conditions recorded at dispatch rather than reconstructed weeks later.

1.2 Motivation

1. **Shrinkage is a custody problem.** Without a per-dispatch expectation, an operator cannot separate a legitimate swap from an unaccounted unit.
2. **Evidence is fragmented.** The proof of delivery, GPS trace, incident call and warehouse count usually live in different tools, so a dispute is expensive to reconstruct.
3. **Settlement has no gate.** Driver pay is released on delivery confirmation rather than on return reconciliation, so the person closest to the variance has no incentive to close it.

These observations produce a single design brief: one record, one seal, one token, one count and one settlement decision. Every feature in the prototype is justified by that brief.

1.3 Problem Statement

The seminar addresses the following problem: to design a browser-delivered control plane that makes the custody of a last-mile consignment verifiable from dispatch to warehouse return without a backend or specialised hardware. The system must:

- bind each consignment to a unique digital seal and a declared buffer quantity;
- permit a seal break only through a customer-authorised, single-use unlock token;
- record telemetry and incident evidence against the same dispatch record;
- reconcile returned buffer stock at unit level and lock settlement on any variance; and
- present one custody state consistently to vendor, driver, customer and auditor.

1.4 Objectives

1. **Model custody explicitly.** Define dispatch, seal and return states as a finite state machine with guarded transitions, so that invalid sequences are impossible by construction.
2. **Issue verifiable identifiers.** Generate collision-checked seals and single-use unlock tokens, and present them as a shareable receipt.
3. **Unify four perspectives.** Implement vendor, driver, customer and audit portals over one reactive store.
4. **Automate reconciliation.** Compute expected return, actual return, variance and damage mismatch, and lock the driver’s settlement on any non-zero variance.

1.5 Scope of the Seminar

The seminar covers requirements, domain modelling, architecture, implementation and evaluation of the prototype, including the custody state machine, the data model, the four portals, the reconciliation algorithm, the KPI layer and the deployment pipeline.

Certified hardware seals, cryptographic key management, server-side authentication, real payment settlement and integration with commercial transport management systems are out of scope; Chapter 6 treats these as limitations. All seeded data is fictional. The source is at `https://github.com/Vivek16-Py/chainguard-v2` and the build at `https://chainguard-v2-ten.vercel.app`.

2. Literature Survey

2.1 Existing Work and Related Studies

Last-mile research maps the actors, activities and outcomes of the final delivery segment. Christopher treats logistics as an integrated, customer-facing system in which value is created or destroyed at organisational interfaces [1]. Olsson, Hellström and Pålsson review the field and note that delivery is increasingly studied as a system rather than a single transport leg [2]. Mangiaracina et al. examine lockers, crowdsourcing and route optimisation for business-to-consumer e-commerce; their objective is efficiency, with cargo integrity treated as a constraint [3].

Warehouse research supplies the counting discipline. Boysen, de Koster and Weidinger survey picking, storage assignment and inventory accuracy in e-commerce warehousing [4]. Their analysis is strong inside the four walls but stops at the dispatch boundary. ChainGuard extends the same accuracy expectation to the buffer stock that travels with the vehicle and returns afterwards.

Traceability research supplies the evidence model. Kshetri argues that blockchain can support traceability, transparency and trust, while noting that adoption depends on trustworthy data entry and governance [5]. Azzi, Chamoun and Sokhn highlight the gap between physical goods and their digital records [6]. ISO 22005 defines traceability principles for food chains [11], and IoT research describes how connected assets make logistics events observable [12], [13]. These sources establish the value of event-level evidence but assume an infrastructure disproportionate to a single fleet.

Access-control research supplies the consent model. NIST SP 800-207 defines zero trust as per-request policy enforcement based on identity and context [7], and the principle of least privilege implies that a customer should authorise one seal break, for one consignment, once [10]. Beck's total retail loss framework broadens the definition of loss beyond theft [8], and Martin cautions that a score can acquire authority its construction does not justify [14]. ChainGuard applies these ideas to a physical custody event and presents its trust-score weights as policy parameters, not measured probabilities.

The literature therefore supplies the components but not the composition: it explains why integration, counting, evidence and consent matter, but not how to bind them to a settlement decision. That gap motivates Chapter 3.

2.2 Summary of Research Papers

Sr.	Authors & year	Paper title / venue	Key focus	Limitation addressed
1	Christopher (2016)	Logistics & Supply Chain Management (Pearson)	End-to-end logistics integration	Integration logic is strategic; no unit-level custody rule.
2	Olsson et al. (2019)	Framework of Last Mile Logistics Research (Sustainability)	Review of actors, activities and outcomes	Reconciliation is an activity, not an invariant.
3	Mangiaracina et al. (2019)	Last-Mile Efficiency in B2C E-commerce (IJPDLM)	Lockers, crowdsourcing, routing	Cargo integrity is a constraint, not a design goal.
4	Boysen et al. (2019)	Warehousing in the E-commerce Era (EJOR)	Warehouse design and inventory accuracy	Analysis stops at dispatch.

Sr.	Authors & year	Paper title / venue	Key focus	Limitation addressed
5	Kshetri (2018)	Blockchain's Roles in SCM (IJIM)	Traceability, transparency and trust	Trust anchors and data-entry integrity unresolved.
6	Azzi et al. (2019)	The Power of a Blockchain-based Supply Chain (CAIE)	Blockchain records for supply chains	Physical-to-digital gap; pilot-scale cost.
7	Rose et al. (2020)	Zero Trust Architecture (NIST SP 800-207)	Per-request policy enforcement	Identity-centric; does not model physical custody.
8	Beck (2016)	Beyond Shrinkage: Total Retail Loss (ECR)	Broad loss taxonomy beyond theft	No last-mile control mechanism.

Table 2.1: Summary matrix of the literature survey

2.3 Research Gaps

1. **No unit-level custody invariant.** Systems identify a consignment and record delivery, but rarely bind seal, authorised break, declared buffer and returned count into one rule that can pass or fail.
2. **Fragmented evidence.** Telemetry, proof of delivery, incident reports and counts are collected by different parties in different formats.
3. **Reconciliation after settlement.** Where reconciliation exists it is a back-office exercise performed after payment, so variance informs an adjustment rather than gating it.
4. **Single-perspective interfaces.** Vendor dashboards, driver apps and customer pages hold separate state, so the same event can appear differently to each role.

3. Overview of the ChainGuard v2 Control Plane

3.1 Definition and Concept

ChainGuard v2 is a custody-verification control plane for last-mile dispatch. It treats the digital seal, rather than the delivery note, as the unit of accountability. A dispatch is created with a core and a buffer quantity; a unique seal is issued; the driver streams telemetry and can raise GPS-anchored incidents; the customer verifies the seal and authorises a break with a single-use token; and the warehouse must return the declared buffer unit-for-unit before settlement is released.

The definition rests on four invariants, enforced in the store rather than in individual screens:

1. **I1 — One active seal per dispatch.** A dispatch starts in `EN_ROUTE_SEALED` with seal state `SEALED`; no second seal is issued.
2. **I2 — Consent before break.** The move to `AWAITING_RETURN` requires the customer attestation, and the generated token is tied to that dispatch.
3. **I3 — Buffer is the expected return.** The expected quantity is the buffer declared at dispatch, not a later estimate.
4. **I4 — Variance gates settlement.** Any non-zero difference sets `payoutLocked`, changes the dispatch to `FRAUD_FLAGGED` and voids the seal.

Because the invariants live in one store, the four portals are views over a single custody record rather than four applications that synchronise after the fact.

3.2 Architecture / Block Diagram

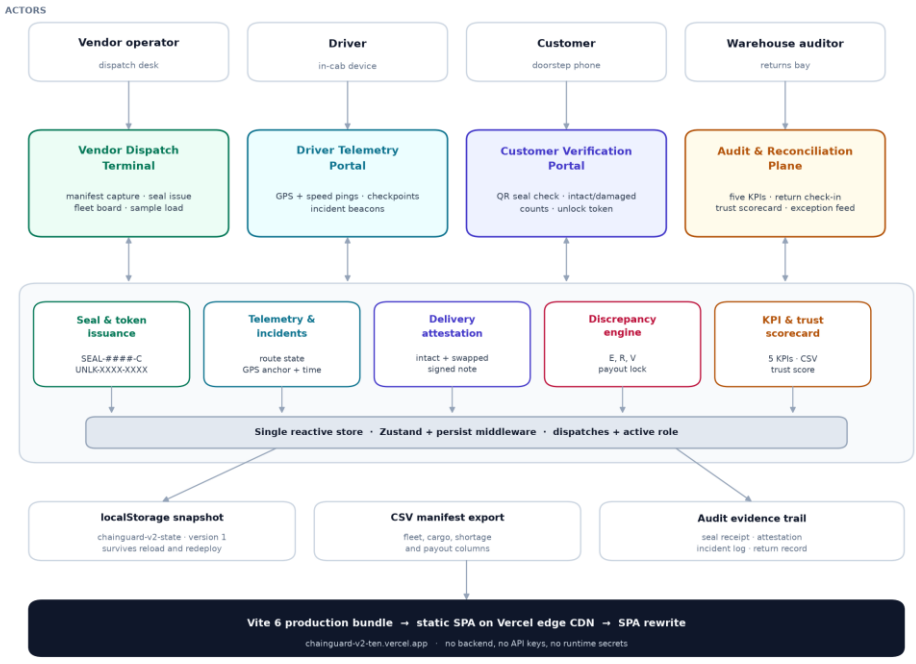


Figure 3.1: Four-plane architecture of the ChainGuard v2 control plane

The architecture has five tiers. The actor tier contains the vendor operator, driver, customer and warehouse auditor. The presentation tier contains one portal per role in a single React application

with a role switch. The domain tier contains five engines — seal and token issuance, telemetry and incidents, delivery attestation, discrepancy detection, and KPI and score computation — around one reactive store. The persistence tier holds the versioned local snapshot and audit exports, and the delivery tier is a static bundle on a content delivery network.

The store uses Zustand and persists a partial snapshot to `localStorage` under `chainguard-v2-state`. All changes are explicit actions: `createDispatch`, `setRouteStatus`, `pingTelemetry`, `advanceCheckpoint`, `logIncident`, `authorizeDelivery`, `checkInReturn` and `resetDemo`. Components read derived values through selectors rather than keeping private copies of custody state. There is no server-side authority and no cryptographic immutability; Chapter 6 treats that boundary explicitly.

3.3 Key Components and Technologies

1. **Vendor Dispatch Terminal.** Captures client, address, quantities, driver, vehicle and route; validates the vehicle format; issues a `SEAL-XXXX-X` seal and a `DSP-####` dispatch; and shows a copyable seal receipt.
2. **Driver Telemetry Portal.** Simulates speed and GPS pings every 3.4 seconds, advances checkpoints and records breakdown, cargo-impact or blockade beacons with GPS and time. The cargo hold is read-only.
3. **Customer Verification Portal.** Shows the seal QR, the shipment summary and the return formula. The customer counts intact and damaged units, adds a mandatory note when swapping, signs and authorises; the store issues a single-use `UNLK-XXXX-XXXX` token.
4. **Audit and Reconciliation Plane.** Combines five KPIs, the fleet and cargo manifest, warehouse check-in, a driver scorecard and an exception feed, with CSV export and a reset control.
5. **Domain model and store.** A `Dispatch` aggregates seal, telemetry, incidents, attestation and return record; optional stages are typed as `delivery?` and `returnRecord?`, and the status unions are closed, so an unsupported state fails compilation.

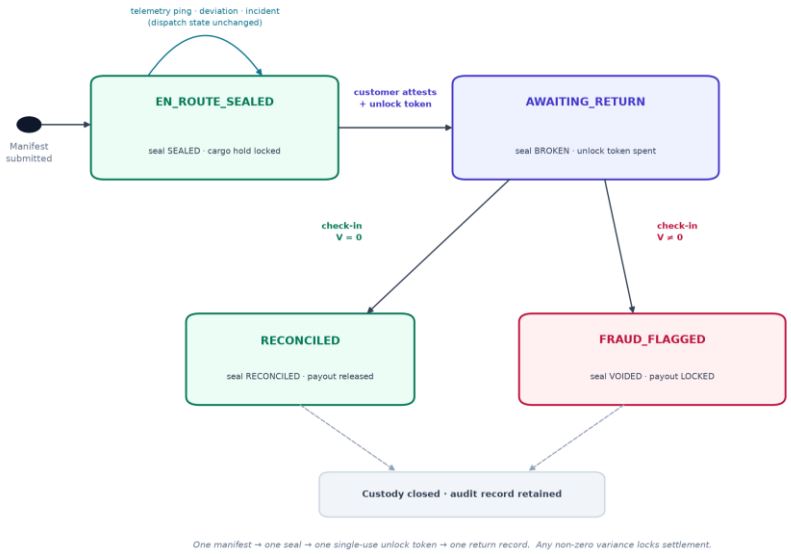


Figure 3.2: Custody lifecycle state machine, guarded transitions and settlement branches

Figure 3.2 summarises the lifecycle implemented by these actions. Telemetry and incidents mutate metadata without changing the custody state; the customer attestation moves a dispatch to awaiting return; and check-in resolves it to reconciled or fraud-flagged, with the seal and payout following the same branch. Table 3.1 lists the guards.

Current state	Trigger	Guard	Next state	Seal state
— (manifest)	createDispatch(manifest)	quantities > 0; vehicle valid	EN_ROUTE_SEALED	SEALED
EN_ROUTE_SEALED	Telemetry ping	dispatch is active	EN_ROUTE_SEALED	SEALED
EN_ROUTE_SEALED	Route status / incident	valid route or incident type	EN_ROUTE_SEALED	SEALED
EN_ROUTE_SEALED	authorizeDelivery()	damaged ≤ buffer; intact ≤ order	AWAITING_RETURN	BROKEN
AWAITING_RETURN	checkInReturn()	V = 0	RECONCILED	RECONCILED
AWAITING_RETURN	checkInReturn()	V ≠ 0	FRAUD_FLAGGED	VOIDED
RECONCILED / FRAUD_FLAGGED	Terminal	none	—	—

Table 3.1: Custody states, guards and permitted transitions

4. Working / Methodology

4.1 Working Principle

The working principle is a linear custody flow with two feedback loops: telemetry and incidents return evidence to the vendor and audit planes while the vehicle is still moving. Figure 4.1 shows the eight stages across the four roles.

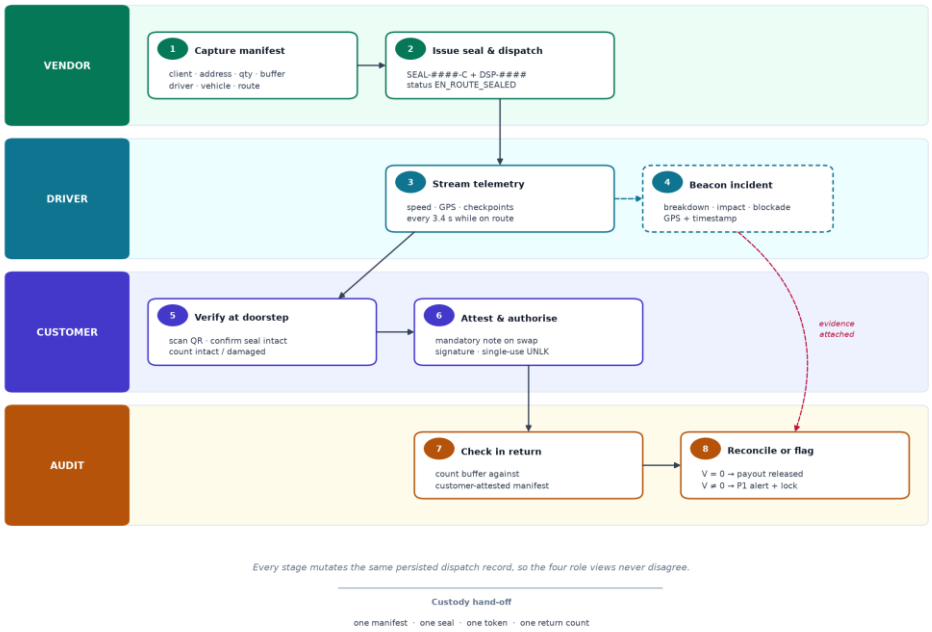


Figure 4.1: End-to-end operational workflow across the four role portals

1. **Manifest capture (vendor).** Client, address, core and buffer quantities, driver, vehicle and route are captured and validated.
2. **Seal issuance (vendor).** The store generates a non-colliding seal, assigns the next dispatch number and creates the record in `EN_ROUTE_SEALED`.
3. **Telemetry (driver).** Speed, GPS and checkpoint updates stream while the dispatch is sealed; production would source these from a telematics unit or driver phone.
4. **Incident beacon (driver).** A breakdown, impact or blockade is recorded with type, note, GPS, time and speed, and appears in the exception feed.
5. **Doorstep verification (customer).** The customer scans the seal QR and counts intact units and damaged units swapped from the buffer.
6. **Authorisation (customer).** A note and signature are required for a swap; authorising issues the unlock token and freezes the expected return at the original buffer.
7. **Return check-in (audit).** The warehouse counts intact and damaged buffer units; the form pre-fills the expectation but records the physical truth.
8. **Closure (audit).** A zero variance reconciles the dispatch and releases payout; any other variance flags fraud, voids the seal, locks payout and raises a priority-1 alert.

Each step is a guarded store mutation, so a portal cannot skip ahead: a driver cannot authorise a break, a customer cannot write a return record, and an auditor cannot reconcile an unattested dispatch.

4.2 Algorithms / Techniques Used

A. Seal and token generation

Seals follow `SEAL-DDDD-C`, with `DDDD` between 1000 and 9999 and `C` drawn from an alphabet that omits `I` and `O`. Generation retries until the candidate is absent from the dispatch list. Dispatch numbers follow `DSP-####` and increment from the highest existing number, with 1042 as the floor. Tokens follow `UNLK-XXXX-XXXX`, each block a hexadecimal value between `0x1000` and `0xEFFF`. The prototype uses `Math.random`, which is adequate for a demonstration but not for a security boundary; production should use a CSPRNG and verify signed tokens server-side with HMAC or Ed25519 [15].

B. The buffer balance law

Let `B` be the declared buffer, `D` the damaged units swapped by the customer, `I` the intact units returned and `R` the total returned. The expected return is the buffer, and the swap is a transfer rather than a write-off:

$$E = B \qquad I_due = B - D \qquad D_due = D$$

$$R = I + D \qquad V = E - R$$

If ten buffer units were loaded and two were swapped, eight intact and two damaged units are due back. The damaged units are returned as evidence, not deducted. A damage mismatch can therefore be reported separately from a count variance: the driver may return the right number of units in the wrong mix. This formula is displayed in the customer portal and frozen into the attestation.

C. Reconciliation and the discrepancy engine

At check-in the audit plane recomputes the expected return from the frozen attestation, counts what was returned and classifies the result:

```
expected  = dispatch.delivery.expectedReturn      // = B
received  = intactReturned + damagedReturned     // = R
variance  = expected − received                  // = V
damageMismatch = damagedReturned ≠ delivery.damagedSwapped
fraud     = variance ≠ 0
status    = fraud ? 'FRAUD_FLAGGED' : 'RECONCILED'
sealState = fraud ? 'VOIDED'        : 'RECONCILED'
payoutLocked = fraud
```

A positive variance is a shortage; a negative variance is an undeclared surplus. Both lock payout. A zero variance with a damage mismatch does not lock payout but attaches a warning, because the count reconciles while the composition does not.

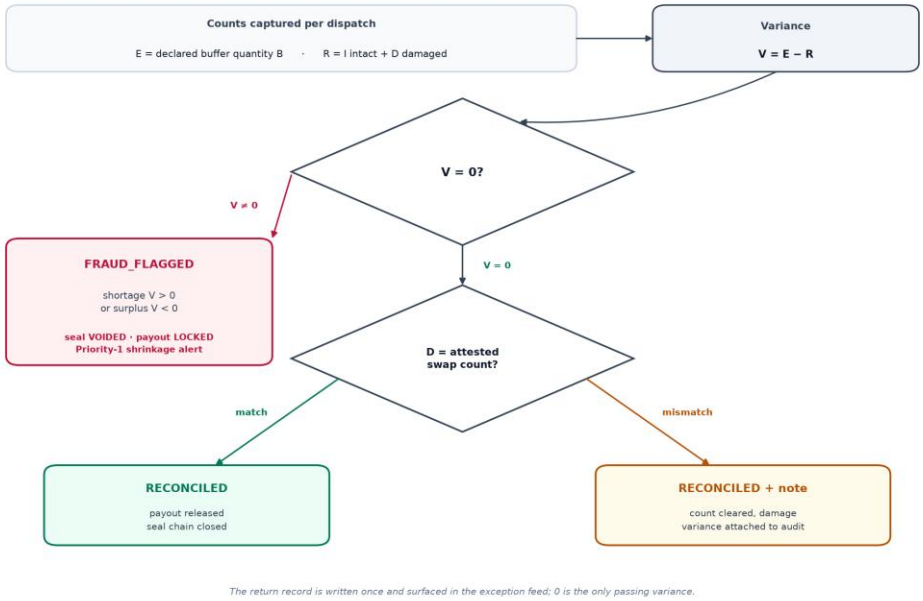


Figure 4.2: Warehouse reconciliation decision flow

The alert is part of the specification because it is where the control plane becomes operationally useful. The interface shows expected, received and variance counts, dispatch and seal identifiers, vehicle, custody breakdown, attested damage and payout state.

> CRITICAL SHRINKAGE DETECTED — Driver [Name] unaccounted for [n] buffer unit(s). Shift settlement payout locked. Telemetry, incident beacons, the customer attestation and the seal token are attached to the settlement record.

D. Fleet KPIs

- **Active dispatches:** `EN_ROUTE_SEALED` or `AWAITING_RETURN`.
- **Sealed en route:** `EN_ROUTE_SEALED` with seal state `SEALED`.
- **Fraud alerts:** dispatches in `FRAUD_FLAGGED`.
- **Value accounted:** returned intact, returned damaged and absolute shortage across completed returns, multiplied by ₹1,200; a documented shortage is evidence, not recovery.
- **Compliance rate:** reconciled divided by completed dispatches, defined as 100 when none are complete.

At the seeded state the dashboard shows four active dispatches, three sealed, one fraud alert, one awaiting return, one reconciled, 50% compliance and ₹24,000 accounted.

E. Driver trust score

The score starts at 100 and applies three penalties, where `M` is missing units, `C` incident beacons and `X` runs with a deviated, stopped or incident route status:

$$T = max(0, min(100, 100 - 22M - 4C - 3X))$$

Scores of 92 or above are Elite, 78–91 Trusted, 60–77 Watchlist and below 60 Flagged; a driver with no runs is Provisioned. The weights are a policy judgement, exposed in one line so an operator can audit them.

F. Evidence export

The exception feed merges fraud events and incident beacons in reverse chronological order; the CSV export writes one row per dispatch with client, route, quantities, seal state, status, shortage and payout flag. Neither is an append-only audit store, but both make the reasoning inspectable.

4.3 Tools and Platforms

Table 4.1 lists the implementation stack; every item is a runtime dependency or a build and delivery component used by the deployed project.

Layer	Component / tool	Functionality
Frontend	React 18, TypeScript 5 (strict), Tailwind CSS 3	Role-based single-page interface with compile-time domain checks
State	Zustand 5 with persist middleware	One reactive store; localStorage key chainguard-v2-state
UI system	Lucide React, custom primitives	Consistent cards, badges, fields, dialogs and toasts
Build	Vite 6, PostCSS, Autoprefixer	Development server and optimised production bundle
Quality gate	tsc --noEmit; npm run build	Type errors and unused symbols fail the build
Deployment	Vercel static hosting, SPA rewrite	Global CDN with automatic deployment from the main branch
Export	Browser Blob API and CSV serialiser	Portable fleet manifest for audit

Table 4.1: Technology stack of the implemented prototype

The seeded fleet in Table 4.2 covers every lifecycle state, so the interface can be evaluated without inventing data and the KPIs in Section 4.2 are reproducible on a fresh load.

Dispatch	Seal	Client / route	Qty	Driver	Status
DSP-1042	SEAL-8921-X	Zenith Retail · BKC	50 + 10	Priya Nair	EN_ROUTE_SEALED
DSP-1041	SEAL-7742-Q	Nova Diagnostics · Airoli	80 + 12	Imran Shaikh	EN_ROUTE_SEALED
DSP-1040	SEAL-6608-M	Harbour Pharma · Nerul	120 + 15	Sameer Qureshi	EN_ROUTE_SEALED
DSP-1039	SEAL-5183-K	Aurora Interiors · Hinjawadi	40 + 8	Rakesh Pawar	AWAITING_RETURN
DSP-1036	SEAL-3304-B	Metro Books · Thane	60 + 10	Anita Deshmukh	RECONCILED
DSP-1030	SEAL-2049-V	Crestline Foods · Mulund	60 + 10	Rakesh Pawar	FRAUD_FLAGGED

Table 4.2: Seeded demonstration fleet (six dispatches)

5. Applications

5.1 Real-World Use Cases

1. **Pharmacy and diagnostics.** A distributor sends 120 test kits with a 15-unit buffer; damaged kits are swapped at the clinic and the remainder returns with a signed attestation, which supports recall and audit queries.
2. **High-value retail.** A single missing unit can exceed the delivery fee; the declared buffer and frozen attestation make the variance explicit and hold settlement until it is explained.
3. **Food and perishables.** A GPS- and time-stamped impact beacon gives the vendor a defensible record that damage occurred in transit rather than in the warehouse.
4. **Third-party fleet settlement.** The scorecard separates a one-off incident from a pattern of shortages, supporting coaching as well as penalties.
5. **Insurance and disputes.** The exception feed and CSV export provide a chronological record assembled at the time of the event rather than weeks later.

5.2 Case Study

The case uses the seeded dispatch DSP-1030. It is a demonstration, not a field deployment. Crestline Foods books 60 core units with a 10-unit buffer, carried by Rakesh Pawar under seal `SEAL-2049-V`. At the doorstep the customer finds two damaged units and swaps them from the buffer, leaving a signed note; the attestation freezes an expected return of ten units, eight intact and two damaged.

At the warehouse the operator counts six intact and two damaged units. Eight have returned against ten expected, so the variance is +2. The engine sets `shortage = 2` and `payoutLocked = true`, changes the dispatch to `FRAUD_FLAGGED` and voids the seal. Because the damaged count matches the attested swap, this is a pure shortage rather than a composition mismatch. At ₹1,200 per unit the documented exposure is ₹2,400.

Rakesh Pawar has two runs: the awaiting DSP-1039 and the failed DSP-1030. With no incidents or deviations but two missing units his score is $100 - (22 \times 2) = 56$, which places him in the Flagged band. Anita Deshmukh, with one clean run, scores 100 and is rated Elite.

The fleet row turns red, the fraud KPI increments, compliance falls to 50%, and value accounted includes the two missing units because they are now documented. The case shows that the swap reduced the intact expectation without changing the total, that the shortage was detected by arithmetic rather than suspicion, and that the payout lock and audit record were produced by the same submission that recorded the count.

6. Advantages and Limitations

6.1 Advantages

- **One invariant replaces four records.** Seal, attestation, telemetry and return count are fields of one dispatch, so stock is reconciled before systems are.
- **Consent is explicit and single-use.** A break requires a customer-issued token visible to the customer, driver and auditor, applying least privilege to a physical event [10].
- **Reconciliation is unit-exact.** Expected return is the declared buffer and variance is computed from counted units, with the same arithmetic driving alert, status and payout.
- **Settlement is gated by the count.** A non-zero variance locks settlement immediately, which makes explaining a discrepancy necessary rather than optional.
- **Evidence is assembled at the event.** Incidents carry GPS and time, attestations carry a note and token, and returns carry operator counts.
- **The four views cannot drift.** Every portal reads one store, so a vendor change is visible in the audit plane without a refresh.
- **Deployment is reproducible.** A static bundle on a CDN with no backend or secrets lets a reviewer reproduce the fraud case in under a minute.

6.2 Limitations / Challenges

- **The trust boundary is the browser.** The role switch is not authentication and `localStorage` can be edited. Production needs server-side identity, role-based access control and an append-only store.
- **Identifiers are not secrets.** Seal and token values come from `Math.random`; production needs a CSPRNG and server-side signature verification [15].
- **No hardware seal or real scan.** The QR panel is rendered, not scanned, so physical tamper evidence is not demonstrated.
- **No backend or multi-device sync.** State is local to one browser; four devices would not share a record.
- **Settlement is a flag, not a payment.** `payoutLocked` stops nothing outside the interface.
- **Weights are policy, not measurement.** The 22, 4 and 3 penalties were chosen for legibility and could create incentives to hide incidents.
- **The buffer is declared, not measured.** A misloaded buffer makes reconciliation consistently wrong; the plane verifies custody against the manifest, not the manifest against the load.
- **Privacy and retention are unresolved.** Real addresses and notes would require minimisation, retention limits and a basis consistent with the DPDP Act, 2023.
- **Evidence is not immutable.** The interface mentions an immutable audit log, but the prototype stores mutable JSON; no hash is computed and no external anchor exists.

The interface honesty point deserves emphasis: the alert footer reads “Evidence hash anchored · immutable audit log”, but no hash chain exists. Before the interface is shown to an auditor, that copy should read “session audit trail” or be backed by a real hash chain. Scale is a second qualification: six dispatches and five drivers are responsive, but KPI and scorecard functions scan the full list on every render and would need indexed queries at fleet scale.

7. Comparison with Existing Technologies

ChainGuard is best positioned against a telematics or transport management system, an electronic proof of delivery and a blockchain traceability platform. Table 7.1 compares them on custody capabilities rather than feature count.

Capability	ChainGuard v2	Telematics / TMS	Paper POD / ePOD	Blockchain traceability
Primary question	What is the custody state and who is accountable?	Where is the vehicle and when will it arrive?	What did the customer sign for?	Can the record be trusted across parties?
Seal identity	Unique SEAL-####-C per dispatch	Not modelled	Signature or photograph	Asset identifier on a ledger
Authorised break	Customer-issued single-use token	Not modelled	Paper signature	Smart-contract condition
Quantity invariant	Buffer returned unit-for-unit	Usually not tracked	Exception note	Tokenised if the data is correct
Reconciliation	Automated; variance locks payout	Manual or back office	Manual	Automated only with a trusted oracle
Settlement gate	Payout locked on variance	Invoice-based	Not applicable	Possible through contract logic
Incident evidence	GPS- and time-stamped beacons	Telematics vendor portal	Call-centre note	On-chain hash, if captured
Infrastructure	Static SPA, no backend	Server, devices and subscriptions	Paper or PDF	Consortium network and governance
Audit immutability	Session-level freeze only	Vendor audit log	Physical archive	Ledger-level, oracle-dependent

Table 7.1: Feature matrix comparison with existing approaches

ChainGuard is narrower than a TMS and deeper than an ePOD. A TMS optimises movement and cost but rarely knows how many buffer units should return; an ePOD captures acceptance without enforcing a quantity invariant or linking acceptance to settlement. ChainGuard occupies the custody decision between arrival and payment.

Blockchain traceability is closest in intent but furthest in infrastructure: a ledger provides integrity once an event is written, but still trusts a party to write the physical count and requires governance to define a valid event [5], [6]. For one fleet, an authenticated hash-chained log delivers most of that assurance at a fraction of the cost; a ledger pays off only when several organisations share one record.

8. Future Scope

1. **Server-side custody service.** Move the store behind an authenticated API with role-based access control, tenant isolation and an append-only event table.
2. **Signed tokens and hardware seals.** Replace random identifiers with CSPRNG tokens signed using HMAC or Ed25519, and integrate an NFC or BLE tamper loop.
3. **Hash-chained audit log.** Roll a hash over each custody event and anchor periodic digests to a timestamping service or permissioned ledger.
4. **Offline-first driver app.** Package the driver portal as an installable progressive web app with queued, signed batch upload.
5. **Settlement integration.** Connect `payoutLocked` to a payout service with an explicit release workflow and exchange data with warehouse and ERP systems.
6. **Risk analytics and governance.** Calibrate the trust score against outcomes, test it for bias [14], and add data minimisation, retention limits and access logging.

9. Conclusion

Last-mile shrinkage is usually treated as a counting problem at the end of a journey. This report has argued that it is a custody problem at the beginning: buffer stock leaves a warehouse under a seal and returns without a rule binding its expected quantity to the evidence of what happened. Without it the final count has no authority and settlement proceeds regardless.

ChainGuard v2 makes the rule executable. Each dispatch binds one seal and one buffer; the seal breaks only with a customer-issued single-use token; telemetry and incidents share the record; and check-in reconciles unit-for-unit, with variance voiding the seal, locking payout and raising a priority-1 alert.

The prototype demonstrates the full workflow, a seeded fleet covering every state and a reproducible fraud case, while stating its limits: client-side trust, non-cryptographic identifiers, settlement as a flag and a mutable audit log. Its broader contribution is a design pattern — one active seal, consent before break, buffer as expected return and variance as a settlement gate — simple to audit and applicable to pharmacy, retail, food and third-party fleets.

REFERENCES

[1] Christopher, M. (2016). Logistics & Supply Chain Management (5th ed.). Harlow: Pearson Education.

[2] Olsson, J., Hellström, D., & Pålsson, H. (2019). Framework of last mile logistics research: A systematic review. Sustainability, 11(24), 7131.

[3] Mangiaracina, R., Perego, A., Seghezzi, A., & Tumino, A. (2019). Innovative solutions to increase last-mile delivery efficiency in B2C e-commerce. International Journal of Physical Distribution & Logistics Management, 49(9), 901–920.

[4] Boysen, N., de Koster, R., & Weidinger, F. (2019). Warehousing in the e-commerce era: A survey. European Journal of Operational Research, 277(2), 396–411.

[5] Kshetri, N. (2018). Blockchain's roles in meeting key supply chain management objectives. International Journal of Information Management, 39, 80–89.

[6] Azzi, R., Chamoun, R. K., & Sokhn, M. (2019). The power of a blockchain-based supply chain. Computers & Industrial Engineering, 135, 582–592.

[7] Rose, S., Borchert, O., Mitchell, S., & Connelly, S. (2020). Zero Trust Architecture (NIST SP 800-207). Gaithersburg, MD: NIST.

[8] Beck, A. (2016). Beyond Shrinkage: Introducing Total Retail Loss. ECR Retail Loss Group.

[9] World Wide Web Consortium. (2022). Verifiable Credentials Data Model 1.1 (W3C Recommendation). https://www.w3.org/TR/vc-data-model/

[10] Saltzer, J. H., & Schroeder, M. D. (1975). The protection of information in computer systems. Proceedings of the IEEE, 63(9), 1278–1308.

[11] International Organization for Standardization. (2007). ISO 22005:2007 — Traceability in the feed and food chain. Geneva: ISO.

[12] DHL Trend Research. (2015). Internet of Things in Logistics. Troisdorf: DHL Customer Solutions & Innovation.

[13] Bouzembrak, Y., Klüche, M., Gavai, A., & Marvin, H. J. P. (2019). Internet of Things in food safety: A review. Trends in Food Science & Technology, 94, 65–74.

[14] Martin, K. (2019). Ethical implications and accountability of algorithms. Journal of Business Ethics, 160, 835–850.

[15] National Institute of Standards and Technology. (2008). The Keyed-Hash Message Authentication Code (HMAC) (FIPS PUB 198-1). Gaithersburg, MD: NIST.

[16] Vivek16-Py. (2026). ChainGuard v2: Last-Mile Integrity Control Plane [Computer software]. https://github.com/Vivek16-Py/chainguard-v2